

Experiment: 01

1. Objectives:

1. To understand how Trapezoidal method works in python.
2. To understand how Simpson's 1/3 method works in python.
3. To understand how Simpson's 3/8 method works in python.

2. Introduction:

Trapezoidal Rule: Approximates the curve by straight lines forming trapeziums.

$$I = \frac{h}{2} [y_0 + 2(y_1 + y_2 + \dots + y_{n-1}) + y_n]$$

Simpson's 1/3 Rule: Approximates the function by parabolas (requires even n).

$$I = \frac{h}{3} [y_0 + 4(y_1 + y_3 + y_5 + \dots) + 2(y_2 + y_4 + y_6 + \dots) + y_n]$$

Simpson's 3/8 Rule: Approximates the curve by cubic polynomials (requires n multiple of 3).

$$I = \frac{3h}{8} [y_0 + 3(y_1 + y_2 + y_4 + y_5 + y_7 + \dots) + 2(y_3 + y_6 + y_9 + \dots) + y_n]$$

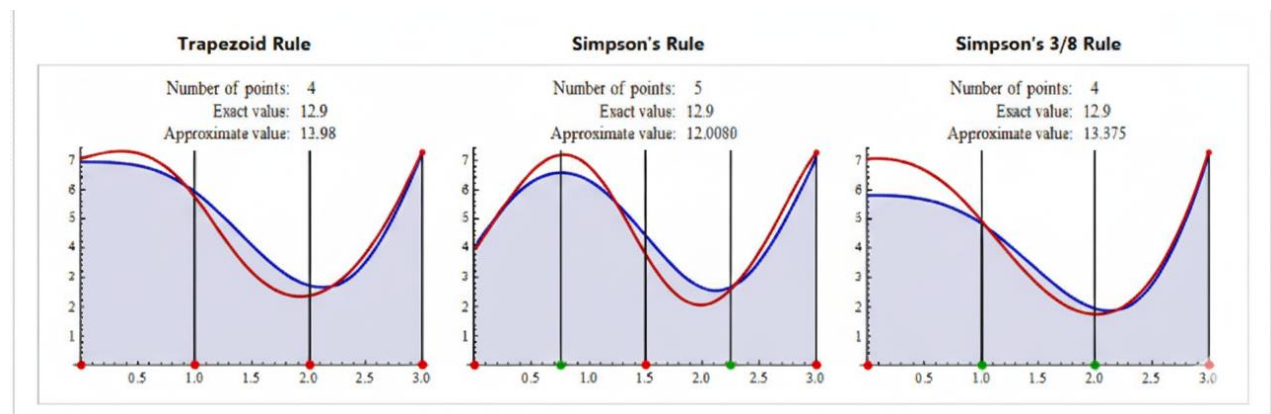


Fig 1: Trapezoidal Rule and Simpson's Rule.

3. Procedure

Algorithm: Integration in Python using Trapezoidal Rule and Simpson's Rule.

1. Start
2. Define the function $f(x) = \frac{1}{1+x^2}$
3. Take input of lower limit x_0 , upper limit x_n , and number of subintervals n .
4. Calculate step size $h = \frac{x_n - x_0}{n}$
5. Compute area using Trapezoidal, Simpson's 1/3, and Simpson's 3/8 formulas.

6. Display the results for comparison.

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** PyCharm Community Edition 20251.1.1

5. Experiment Implementation:

Code:

```
def f(x):  
    return 1 / (1 + x * x)  
  
def Trapezoidal_rule(x0, xn, n):  
    h = (xn - x0) / n  
    area = f(x0) + f(xn)  
  
    for i in range(1, n):  
        area += 2 * f(x0 + i * h)  
  
    area *= h / 2  
    return area  
  
def Simpson_1by3_rule(x0, xn, n):  
    if n % 2 != 0:  
        print("n must be even for Simpson's 1/3 Rule.")  
        return None  
  
    h = (xn - x0) / n  
    area = f(x0) + f(xn)  
  
    for i in range(1, n):  
        if i % 2 == 0:  
            area += 2 * f(x0 + i * h)  
        else:  
            area += 4 * f(x0 + i * h)  
  
    area *= h / 3  
    return area
```

```

def Simpson_3by8_rule(x0, xn, n):
    if n % 3 != 0:
        print("n must be a multiple of 3 for Simpson's 3/8
Rule.")
        return None

    h = (xn - x0) / n
    area = f(x0) + f(xn)

    for i in range(1, n):
        if i % 3 == 0:
            area += 2 * f(x0 + i * h)
        else:
            area += 3 * f(x0 + i * h)

    area *= (3 * h) / 8
    return area

a, b, n = 0, 1, 6

area1 = Trapezoidal_rule(a, b, n)
area2 = Simpson_1by3_rule(a, b, 6)
area3 = Simpson_3by8_rule(a, b, 6)

print ("\nNumerical Integration")
print(f"Exact Trapezoidal Area : {area1:.6f}")
print(f"Exact Simpson 1/3 Area : {area2:.6f}")
print(f"Exact Simpson 3/8 Area : {area3:.6f}")

```

Output:

```

Numerical Integration
Exact Trapezoidal Area : 0.784241
Exact Simpson 1/3 Area : 0.785398
Exact Simpson 3/8 Area : 0.785396

```

6. Discussion & Conclusion:

In this experiment we have used Trapezoidal, Simpson's $1/3$, and Simpson's $3/8$ methods to get the accurate results for the given function. However, Simpson's methods, which use higher-order polynomial interpolation, produced slightly more precise values compared to the Trapezoidal rule. Among them, Simpson's $3/8$ rule showed the best performance when the number of intervals was a multiple of three. Overall, these numerical integration techniques effectively approximate definite integrals where analytical solutions are complex or unavailable.

7. Reference:

1. https://www.researchgate.net/figure/Mathematica-application-for-numerical-integration_fig1_273578880